Sprint 2 Contribution Log - Kaila

Overview

In Sprint 2, I worked on the database backend (SQLAlchemy, Alembic, PostgreSQL on Neon). I completed five tasks: KA-1 (profile, resume file and job tables with the migration setup), SA-4 (applications table), KA-2 (sample-jobs script), KA-3 (resume storage functions) and KA-4 (job search). This log is updated as Sprint 2 continues.

KA-1 - Profile, Resume File and Job Tables

Goal: Add three new tables and the migration tooling for the database.

What I did:

Profile model: SQLAlchemy model for user profile information (App/models/profile.py).

ResumeFile model: SQLAlchemy model for uploaded resumes (App/models/resumeFile.py).

Job model: SQLAlchemy model for job postings (App/models/job.py).

Relationships: foreign keys and constraints linking the new models to the existing users table.

Alembic setup: configured Alembic to track and apply database schema changes (alembic/env.py).

Initial migration: generated the initial database schema migration (alembic/versions/e3be2c270f74_initial_database_schema.py).

SA-4 - Applications Table

Goal: Add the applications table, which records a job seeker applying to a job.

What I did:

Application model: SQLAlchemy model linking a user, a job and (optionally) the resume used, with a status and an application date (App/models/application.py).

Data rules: a unique rule on user and job so a user can apply to a job only once. Deleting a user or job removes their applications, and deleting a resume keeps the application but clears the link.

Model registration: imported the model so Alembic detects it (App/models/__init__.py).

Migration: generated a migration that builds on KA-1 and creates only the new table (alembic/versions/3f0afa113594_add_applications_table.py).

KA-2 - Sample-Jobs Script

Goal: One command that fills the database with sample job postings so the team can test search and applications.

What I did:

Seed script: 18 sample jobs, with 3 in each of 6 categories and 3 in each of 6 locations, 5 titles containing "Developer" and 1 closed job. Sample jobs have no recruiter (scripts/seed_jobs.py).

No duplicates: the script skips any job with the same title and company, so it is safe to run more than once.

README: added a setup step explaining how to load the sample jobs, in a separate pull request.

KA-3 - Resume Storage Functions

Goal: Database functions for storing resumes, so the upload endpoints only need to call them.

What I did:

Resume functions: save, list, get, replace and delete. Every function filters by the owner's user id, so a user can never reach someone else's resume (App/crud/resumes.py).

Performance: listing resumes does not load the file data, so it stays fast.

Upload limits: a 5 MB maximum and the allowed file types (PDF, .doc and .docx), shared with the backend and frontend (App/resume_rules.py).

KA-4 - Job Search Query Function

Goal: Database functions behind job search, so the search endpoints only need to call them.

What I did:

Job search: filters by keyword (title, company or description), location and category, ignoring case. Filters that are left out are skipped, and all filters given must match (App/crud/jobs.py).

Paging and order: returns one page of results, newest first, plus the total number of matches counted before paging.

Closed jobs: hidden by default, with an option to include them for recruiter views.

Job lookup: returns one job by id, or nothing if it doesn't exist.

Verification

All five tasks are verified against Neon: the migrations match the database, the seed script adds 18 jobs without duplicates, the resume functions passed 12 of 12 checks and the job search passed 16 of 16. Running the full migration chain on a fresh database is still untested.

KA-1: Inspected the existing PostgreSQL database structure, tested that the new models import without errors, ran Alembic's schema check to confirm the models match the existing tables, and generated an SQL preview of the migration.

KA-1 on Neon: Found that Neon had no Alembic version recorded, and that the KA-1 migration also creates the users table, which already existed, so running it would have failed. I confirmed in pgAdmin that all four KA-1 tables already existed, then ran alembic stamp head so Alembic records the migration as applied without changing any tables.

SA-4: Checked the migration file: it builds on KA-1, creates only the applications table, includes the unique rule, and its downgrade drops only that table. Applied it with alembic upgrade head (now at 3f0afa113594) and confirmed the table in pgAdmin. After pulling the latest main, ran alembic check to confirm a teammate's change to the user model needed no new migration.

KA-2: Ran the seed script twice. The first run added 18 jobs and the second added none.

KA-3: Ran a throwaway check with two real users covering save, list, get, replace and delete. Every attempt by the second user to read, change or delete the first user's resume was refused, and the test data was removed afterwards.

KA-4: Ran a read-only check against the sample jobs. No filters returned 17 open jobs with the closed one hidden, "developer" returned 7 jobs that all contain the word, Montreal plus Software Development returned 1, and an unknown keyword returned an empty list with a total of 0. Paging, newest-first order and job lookup also passed.

Work Log

Oct 13, 2026 - KA-1, SA-4, KA-2, KA-3, KA-4: Recorded the KA-1 migration as applied on Neon (alembic stamp head). Built the applications model and migration, applied it to Neon and opened a pull request. Wrote the sample-jobs seed script and README step. Wrote and tested the resume storage functions (12 of 12 checks) and the job search functions (16 of 16 checks), and opened pull requests for each. Hours: to add.

Week of Oct 8, 2026 - KA-1: Built the Profile, ResumeFile and Job models, set up Alembic, created the initial migration, verified it, and merged the pull request. Hours: to add.